

Data Structures & Algorithms Internship Assignment

General Instructions

This assignment is intended to evaluate your problem-solving skills, coding standards, algorithmic thinking, and ability to write maintainable software. **Submission Rules** Complete all 10 questions. Write clean, modular, and readable code. Do not copy solutions from the Internet or use AI-generated code. Use meaningful variable and function names. Validate every user input. Do not use built-in sorting/searching methods where manual implementation is required. Every question must be implemented as a separate program or module. Include comments explaining complex logic. Prepare a README explaining how to compile and run each program. Push the project to GitHub with regular commits instead of one final commit. **For EVERY Question Include:** Problem Understanding (2-3 lines) Algorithm or Pseudocode Source Code Sample Input Sample Output At least 5 custom test cases Time Complexity Space Complexity Limitations (if any) **Coding Standards** Follow consistent indentation. No duplicated code. Break large logic into reusable methods. Avoid magic numbers. Handle invalid inputs gracefully.

Easy 1 – Number Statistics

Objective: Read N integers and calculate sum, average, minimum, maximum, even count, odd count, positive count and negative count.

Requirements Design the solution before coding. Validate all inputs. Display user-friendly messages. Analyse time and space complexity. Provide at least 5 test cases including edge cases.

Easy 2 – Number Properties

Objective: Determine whether a number is Prime, Palindrome and Armstrong. Display all applicable properties.

Requirements Design the solution before coding. Validate all inputs. Display user-friendly messages. Analyse time and space complexity. Provide at least 5 test cases including edge cases.

Easy 3 – Pattern Generator

Objective: Generate right triangle, inverted triangle, pyramid and diamond using nested loops.

Requirements Design the solution before coding. Validate all inputs. Display user-friendly messages. Analyse time and space complexity. Provide at least 5 test cases including edge cases.

Medium 4 – Bubble Sort Visualizer

Objective: Implement Bubble Sort manually and display the array after every pass. Count comparisons and swaps.

Requirements Design the solution before coding. Validate all inputs. Display user-friendly messages. Analyse time and space complexity. Provide at least 5 test cases including edge cases.

Medium 5 – Searching Algorithms

Objective: Implement Linear Search and Binary Search. Compare execution steps and explain when Binary Search should be used.

Requirements Design the solution before coding. Validate all inputs. Display user-friendly messages. Analyse time and space complexity. Provide at least 5 test cases including edge cases.

Medium 6 – Matrix Operations

Objective: Perform matrix addition, multiplication, transpose, row sums, column sums, diagonal sums, identity and symmetry checks.

Requirements Design the solution before coding. Validate all inputs. Display user-friendly messages. Analyse time and space complexity. Provide at least 5 test cases including edge cases.

Medium 7 – Student Record Management

Objective: Store student ID, Name and Marks. Support Add, Update, Delete, Search, Display and Sort by Name or Marks.

Requirements Design the solution before coding. Validate all inputs. Display user-friendly messages. Analyse time and space complexity. Provide at least 5 test cases including edge cases.

Hard 8 – Matrix Rotation

Objective: Rotate an NxN matrix by 90°, 180° and 270° without using helper libraries. Verify output with test cases.

Requirements Design the solution before coding. Validate all inputs. Display user-friendly messages. Analyse time and space complexity. Provide at least 5 test cases including edge cases.

Hard 9 – Frequency Analyzer

Objective: Count frequency of every element, identify duplicates, unique values, most frequent and least frequent elements.

Requirements Design the solution before coding. Validate all inputs. Display user-friendly messages. Analyse time and space complexity. Provide at least 5 test cases including edge cases.

Hard 10 – Banking Management System

Objective: Menu-driven application supporting account creation, deposit, withdrawal, transfer, balance inquiry and transaction history with proper validation.

Requirements Design the solution before coding. Validate all inputs. Display user-friendly messages. Analyse time and space complexity. Provide at least 5 test cases including edge cases.

Evaluation Criteria (100 Marks)

Correctness - 40 Marks

Code Quality & Readability - 15 Marks

Algorithm Design - 15 Marks

Complexity Analysis - 10 Marks

Input Validation - 5 Marks

Test Cases - 10 Marks

Documentation & Git History - 5 Marks